Residential Solar PV & Battery Optimizer Code Walkthrough

Contents

1 Section 1 – Imports (lines 61–74) 2

2 Section 2 – File Paths, Catalogs, and Constants (lines 81–223) 3
- 2.1 2A – File Paths and Geographic Constants (lines 84–101) . . . 3
- 2.2 2B – Solar Panel Catalog (lines 106–152) . . . 3
- 2.3 2C – Battery Catalog (lines 157–183) . . . 4
- 2.4 2D – Physical and Financial Constants (lines 185–214) . . . 5
- 2.5 2E – MILP Battery Physics Constants (lines 209–214) . . . 5

3 Section 3 – USER_PROFILE (lines 242–266) 6

4 Section 4A – `_load_household_profile_from_eia()` (lines 275–445) 7
- 4.1 Steps 1 & 2 – Load, Downscale, 5-Year Average by HOY . . . 7
- 4.2 Step 3 – Nine Variability Factors . . . 7
- 4.3 Step 5 – Rescale to Known Annual Total . . . 8

5 Section 4B – Synthetic Load Fallback (lines 465–486) 9

6 Section 4C – Irradiance (lines 491–525) 10

7 Section 4D – SDG&E TOU Tariff Builder (lines 530–578) 10

8 Section 4E & 4F – PV Output and Hardware Selection (lines 583–620) 11

9 Section 4G – Rule-Based Dispatch Simulation (lines 625–673) 11

10 Section 4H – 10-Year NPV Model (lines 678–746) 12

11 Section 5 – Core MILP Optimizer (lines 753–981) 14
- 11.1 Variable Layout . . . 14
- 11.2 Objective Function . . . 14
- 11.3 All 10 Constraint Groups . . . 15

12 Section 6 – Master Orchestrator (lines 988–1097) 16

13 Section 7 – Output Assembly: `acfg()` Full Array Configuration 17
- 13.1 Full `acfg()` Function . . . 17
- 13.2 Mathematical Foundation . . . 18
  - 13.2.1 Panel Dimension Estimation from Area . . . 18
  - 13.2.2 Physical Layout: Rows and Columns on the Roof . . . 19
  - 13.2.3 Cell Topology Inside Each Panel . . . 19
  - 13.2.4 Electrical String Wiring . . . 20
- 13.3 Full Hierarchy and Output . . . 20

Section 1 – Imports (lines 61–74)

Lines 61–74

```python
import hashlib
import math
import os
import time
import warnings
from datetime import datetime, timedelta

import numpy as np
import pandas as pd
import requests
from scipy.optimize import milp, LinearConstraint, Bounds
from scipy.sparse import csc_matrix

warnings.filterwarnings("ignore")
```

Import Roles

Module	Role
`hashlib`	SHA-256 of `"lat_lon"` string to a deterministic 32-bit integer seed; identical location always produces identical variability factors
`math`	`sqrt()` for Euclidean city-centre distance (F5) and roof-dimension fallback inside `acfg()`
`os`	`os.path.abspath(__file__)` locates CSV data files relative to the script
`time`	Wall-clock timing of each MILP solve
`numpy`	Vectorised 8760-element array arithmetic; COO sparse matrix assembly
`pandas`	Read TOU price CSVs; UTC → America/Los_Angeles timezone conversion
`requests`	HTTP GET to Open-Meteo 5-year archive API
`scipy.milp`	HiGHS MILP solver – the core engine of the whole script
`LinearConstraint`	Wraps the CSC constraint matrix A with per-row bounds $[\ell_i, u_i]$
`Bounds`	Per-variable lower/upper bounds vector
`csc_matrix`	Compressed Sparse Column – HiGHS native format; reduces RAM from ∼500 MB to ∼10 MB

Section 2 – File Paths, Catalogs, and Constants (lines 81–223)

2A – File Paths and Geographic Constants (lines 84–101)

Lines 84–101

```
_HERE = os.path.dirname(os.path.abspath(__file__))

TOU_DR_PATH  = r"C:\Users\shubh\...\tou_dr_daily_2021_2025.csv"
TOU_DR1_PATH = r"C:\Users\shubh\...\tou_dr1_daily_2021_2025.csv"
TOU_DR2_PATH = r"C:\Users\shubh\...\tou_dr2_daily_2021_2025.csv"
EIA_LOAD_PATH = r"C:\Users\shubh\...\San_Diego_Load_EIA_Fixed.csv"

SDGE_TOTAL_CUSTOMERS = 1_040_149   # all SDG&E meters incl. commercial
COASTAL_LON_REF      = -117.25     # coastal / inland split longitude
CITY_CENTER_LAT      =  32.7157    # downtown San Diego
CITY_CENTER_LON      = -117.1611
DEFAULT_RATE_PLAN    = "TOU_DR"
```

EIA Per-Household Downscale

The EIA CSV records hourly grid-wide MW. Dividing by the total meter count converts to per-household average kW:

$$\bar{P}_{\text{household}}(t) = \frac{P_{\text{grid}}(t)\ [\text{MW}] \times 1000}{N_{\text{customers}}} \quad [\text{kW}]$$

Because the denominator includes commercial and industrial meters, the raw per-household number is much higher than a real home. The `annual_consumption_kwh` override in `USER_PROFILE` corrects the magnitude while preserving the temporal shape.

2B – Solar Panel Catalog (lines 106–152)

Lines 106–152 (2 of 9 panels shown)

```
SOLAR_PANEL_CATALOG = [
  {"manufacturer": "REC Group",   "model": "Alpha Pure",
   "efficiency_percent": 22.6,    "cost_per_wp_usd": 2.85,
   "temp_coeff_pct_per_C": -0.26,
   "voc_v": 43.2, "isc_a": 13.8, "vmp_v": 36.4, "imp_a": 13.1,
   "panel_power_w": 405, "panel_area_m2": 1.79,
   "cells_in_series": 60, "cells_in_parallel": 1,
   "degradation_rate": 0.005},
  # JA Solar DeepBlue, Trina Vertex S, Canadian TOPHiKu7, Silfab Prime,
  # Jinko Tiger Neo, LONGi Hi-MO 6, Maxeon 7, Aiko Neostar 2P
]
```

Panel Catalog – Key Relationships

All values at STC ($G_{\text{ref}} = 1000\,\text{W/m}^2$, $T_c = 25°\text{C}$).
Cell-level quantities (computed in Section 7):

$$P_c = \frac{P_{\text{panel}}}{n_s n_p}, \quad V_c = \frac{V_{\text{mp}}}{n_s}, \quad I_c = \frac{P_c}{V_c}, \quad V_{\text{oc},c} = \frac{V_{\text{oc}}}{n_s}, \quad I_{\text{sc},c} = \frac{I_{\text{sc}}}{n_p}$$

Power degradation over lifetime year y:

$$P(y) = P_0(1-d)^{y-1}, \quad d = 0.005 \text{ (REC Alpha Pure)}$$

Auto-selection when `panel_brand = None`:

$$\text{panel}^* = \arg\max_p \frac{\eta_p\ [\%]}{c_p\ [\$/\text{Wp}]} \quad \Rightarrow \quad \text{REC Alpha Pure: } \frac{22.6}{2.85} = 7.93 \text{ (highest ratio)}$$

Manufacturer	**Model**	η (%)	**\$/Wp**	η/c	n_s	**W**
REC Group	Alpha Pure	22.6	2.85	**7.93**	60	405
JA Solar	DeepBlue	21.5	2.90	7.41	60	395
Trina Solar	Vertex S	21.8	2.90	7.52	60	400
Canadian Solar	TOPHiKu7	22.5	3.10	7.26	66	420
Silfab Solar	Prime	22.1	3.15	7.02	60	410
Jinko Solar	Tiger Neo	23.8	3.20	7.44	66	440
LONGi Solar	Hi-MO 6	23.3	3.35	6.96	66	435
Maxeon Solar	Maxeon 7	22.8	3.50	6.51	66	430
Aiko Solar	Neostar 2P	24.3	3.75	6.48	66	460

2C – Battery Catalog (lines 157–183)

Lines 157–183

```
BATTERY_CATALOG = [
  {"manufacturer":"Tesla",   "model":"Powerwall 3",
   "usable_capacity_kwh":13.5, "nominal_voltage_v":50.0,
   "max_charge_power_kw":11.5, "max_discharge_power_kw":11.5,
   "round_trip_efficiency_pct":97.5, "cycle_life":4000,
   "cost_usd":11500, "annual_degradation_rate":0.010},
  # Enphase IQ 5P (5.0 kWh, $6,000)
  # Generac PWRcell M6 (9.0 kWh, $10,000)
  # SolarEdge Home 48V (9.7 kWh, $9,500)
  # Panasonic EverVolt H (17.1 kWh, $15,000)
]
```

Table 1: Residential Battery Catalog Used in the Optimizer

Manufacturer	Model	Capacity (kWh)	Voltage (V)	Charge (kW)	Discharge (kW)	Eff. (%)	Cycle Life	Cost (USD)
Tesla	Powerwall 3	13.5	50.0	11.5	11.5	97.5	4000	11500
Enphase	IQ Battery 5P	5.0	48.0	3.84	3.84	96.0	4000	6000
Generac	PWRcell M6	9.0	48.0	6.7	6.7	96.5	3500	10000
SolarEdge	Home Battery 48V	9.7	48.0	5.0	5.0	94.5	6000	9500
Panasonic	EverVolt H Series	17.1	51.8	7.6	7.6	97.0	6000	15000

Battery Selection Logic ( )

$$\text{battery}^* = \arg\min_{b \in \mathcal{B}'} \frac{c_b\ [\$]}{C_b\ [\text{kWh}]}, \qquad \mathcal{B}' = \{b \in \text{catalog} : C_b \geq B^*\}$$

If no single unit covers B^*, fall back to stacking the largest: units $= \lceil B^*/C_{\text{unit}} \rceil$.

For this run: $B^* = 9.999\,\text{kWh}$. Candidates with $C \geq 9.999$: Powerwall 3 ($852/kWh), SolarEdge ($979/kWh), Panasonic ($877/kWh). Powerwall 3 wins; 1 unit required.

2D – Physical and Financial Constants (lines 185–214)

Lines 185–214

```
PR_PERFORMANCE_RATIO      = 0.80     # soiling + wiring + mismatch + inverter
INSTALLATION_COST_RATE    = 0.10     # 10 pct labour on hardware cost
FEDERAL_ITC_RATE          = 0.30     # 30 pct Federal ITC (2025)
UTILITY_INFLATION_RATE    = 0.06     # SDG&E annual escalation
DISCOUNT_RATE             = 0.07     # NPV discount rate
O_AND_M_COST_PER_W_YR     = 0.005    # cleaning + insurance ($/W/yr)
NEM_EXPORT_CREDIT         = 0.10     # NEM 3.0 avoided-cost ($/kWh)
INVERTER_REPLACEMENT_USD  = 2000     # year-10 inverter swap
ANALYSIS_YEARS            = 10
SDGE_DAILY_FIXED_FEE      = 0.345    # $/day minimum charge

_r = 0.07; _n = 10
ANNUITY_FACTOR = _r*(1+_r)**_n / ((1+_r)**_n - 1)    # = 0.1424
BAT_COST_PER_KWH_PROXY    = 852.0    # $11,500 / 13.5 kWh (Powerwall 3)
```

Financial Formulas from These Constants

Annuity factor – converts lump-sum CAPEX to equivalent $/yr:

$$\alpha = \frac{r(1+r)^n}{(1+r)^n - 1} = \frac{0.07 \times 1.07^{10}}{1.07^{10} - 1} = 0.1424$$

Net CAPEX after installation markup and ITC:

$$C_{\text{net}} = \underbrace{(C_{\text{PV}} + C_{\text{bat}}) \times 1.10}_{\text{gross incl. labour}} \times \underbrace{0.70}_{\text{after 30\% ITC}}$$

Performance ratio decomposition:

$$\text{PR} = 0.80 \approx (1-0.04)_{\text{soil}} \cdot (1-0.03)_{\text{wire}} \cdot (1-0.03)_{\text{mismatch}} \cdot (1-0.10)_{\text{inverter}}$$

2E – MILP Battery Physics Constants (lines 209–214)

Battery Physics in the MILP

Constant	Value	Used in	Meaning
MILP_ETA_C	0.975	C2	Charge efficiency η_c
MILP_ETA_D	0.975	C2	Discharge efficiency η_d
MILP_C_RATE	0.50	C5, C6	Max power = 50% capacity/h
MILP_SOC_MIN_FRAC	0.10	C4	Depth-of-discharge floor
MILP_SOC_INIT_FRAC	0.50	C0	Initial SOC = 50% of B

Section 3 – USER_PROFILE (lines 242–266)

Lines 242–266

```
USER_PROFILE = {
    # Location -- drives all 9 EIA variability factors
    "latitude":                32.7157,
    "longitude":              -117.1611,

    # Strongly recommended: actual kWh from your SDG&E bill
    "annual_consumption_kwh": 18000.0,

    # Financial + physical constraints
    "budget_usd":              50000.0,
    "roof_area_m2":            15.0,
    "roof_length_m":           3.0,     # NEW: longer axis (ridge-to-eave)
    "roof_width_m":            5.0,     # NEW: shorter axis (along the ridge)

    # SDG&E rate plan
    "rate_plan":               "TOU_DR",  # "TOU_DR" | "TOU_DR1" | "TOU_DR2"

    # None = auto-select by best efficiency/cost ratio
    "panel_brand":             None,

    # Only used by synthetic load fallback (if EIA file missing)
    "num_evs":                 0,
    "num_people":              3,
    "num_daytime_occupants":   1,
}
```

Why `annual_consumption_kwh` is critical

The EIA CSV aggregates all SDG&E meter types (residential, commercial, industrial). Dividing by $N_{\text{customers}} = 1{,}040{,}149$ still yields ~17,000+ kWh/yr per "meter" – far too high for a single home. Setting `annual_consumption_kwh = 7500.0` preserves the real temporal *shape* of the EIA load (peak/trough hours) while rescaling the *magnitude* to match the actual SDG&E bill. Set to `None` only to see the raw inflated value.

New keys: `roof_length_m` and `roof_width_m`

They feed `acfg()` so it can compute how many panels physically fit on the roof in each orientation. Their product must equal `roof_area_m2`: $10.0 \times 5.0 = 50.0\,\text{m}^2$. If either is `None`, `acfg()` falls back to $\ell = \sqrt{2A}$, $w = \sqrt{A/2}$ (2:1 aspect ratio).

Section 4A – `_load_household_profile_from_eia()` (lines 275–445)

Purpose

Convert five years of EIA regional MW load data into a single realistic 8760-h per-household kW profile using 5-year averaging, 9 location factors, deterministic noise, and magnitude rescaling.

Steps 1 & 2 – Load, Downscale, 5-Year Average by HOY

Lines 328–346

```
df = pd.read_csv(EIA_LOAD_PATH)
df["dt_local"] = df["dt_utc"].dt.tz_localize("UTC") \
                                .dt.tz_convert("America/Los_Angeles")
df["kw"]  = df["MW_Load"] * 1000.0 / SDGE_TOTAL_CUSTOMERS
df["hoy"] = ((df["doy"] - 1) * 24 + df["hour"]).clip(0, 8759)

# Average all full years (>= 8000 rows) at each HOY position
full_years = df["year"].value_counts()
full_years = full_years[full_years >= 8000].index.tolist()
avg_hoy = (df[df["year"].isin(full_years)]
            .groupby("hoy")["kw"].mean()
            .reindex(range(8760))
            .interpolate("linear").ffill().bfill())
profile = avg_hoy.values.copy()   # shape (8760,)
```

Downscale and HOY Averaging

$$P_{\text{hh}}(t) = \frac{P_{\text{grid}}(t)\,[\text{MW}] \times 1000}{N_{\text{cust}}}, \qquad \text{HOY}(t) = (\text{doy} - 1) \times 24 + \text{hour}$$

$$\bar{P}[h] = \frac{1}{|Y|} \sum_{y \in Y} P_y[h], \qquad Y = \{y : \text{rows}(y) \geq 8000\}$$

The threshold ≥ 8000 (not ≥ 8760) is critical: DST removes one clock hour per year, so no single year ever reaches 8760 rows. The old ≥ 8760 check always failed, falling back to only the 579-h partial 2026 data tiled 15×, inflating the load to ∼24,000 kWh/yr. Averaging 5 full years covers $\frac{8644}{8760} = 98.7\%$ of HOY slots; the remaining 116 DST-gap slots are filled by linear interpolation.

Step 3 – Nine Variability Factors

Lines 348–431 (factor assignments)

```
loc_seed = int(hashlib.sha256(f"{lat}_{lon}".encode())
                .hexdigest(), 16) % (2**32)
rng_hh  = np.random.RandomState(loc_seed)        # F4
rng_sol = np.random.RandomState(loc_seed+1000)    # F7
rng_ev  = np.random.RandomState(loc_seed+2000)    # F8
rng_mg  = np.random.RandomState(loc_seed+3000)    # F9

# F1 Longitude (coastal <-> inland)
dc = longitude - COASTAL_LON_REF      # delta from -117.25
f1 = 1.25 if dc>=0.15 else (1.05+(dc-0.10)*4 if dc>=0.10 else
     0.95+dc if dc>=0 else 0.90+(dc+0.05) if dc>=-0.05 else 0.85)

# F2 Latitude step function
f2 = 1.10 if lat<32.60 else 1.05 if lat<32.70 else \
```

```
15      1.00 if lat<32.85 else 0.95 if lat<32.95 else 0.90
16
17 # F3 Elevation proxy
18 f3 = 1.0 + (max(0,lon-COASTAL_LON_REF) + max(0,lat-32.70)*2)*0.15
19
20 # F4 Household size x appliance efficiency (Deterministic)
21 f4 = 0.70 + 0.10 * min(num_people, 6)
22
23 # F5 Neighbourhood density (Euclidean dist to downtown)
24 f5 = 0.7 if dist<0.03 else 0.9 if dist<0.08 else 1.1 if dist<0.15 else 1.3
25
26 # F6 Economic / home age
27 f6 = (1.15 if coastal+north else 1.05 if coastal else
28       1.25 if urban_core else 0.95 if lon>-117.00 else 1.10)
29
30 # F9 Multi-generational (scalar, applied before time-dependent factors)
31 mg_prob = 0.25 if (south and urban) else 0.15 if urban else 0.10
32 f9 = U(1.20, 1.50) if rng_mg.random()<mg_prob else 1.0
33
34 profile *= f1 * f2 * f3 * f4 * f5 * f6 * f9
35
36 # F7 Solar adoption (time-dependent sine midday reduction)
37 # F8 EV charging (Deterministic time-dependent overnight addition)
38 if num_evs > 0:
39     # 7.2 kW Level 2 charging added for required hours
40     profile += np.where(is_chg, 7.2 * num_evs, 0.0)
41
42 # F10 Daytime Occupants (Deterministic mask 9 AM - 5 PM)
43 if num_daytime_occupants > 0:
44     daytime_mask = (hrs >= 9) & (hrs < 17)
45     profile = np.where(daytime_mask, profile * (1.0 + 0.05 *
           num_daytime_occupants), profile)
```

Factor Composition and Definitions

$$P_{\text{scaled}}(t) = \bar{P}(t) \cdot F_1 F_2 F_3 F_4 F_5 F_6 F_9 \cdot [1 - F_7(t)] + F_8(t) \ \pm 3\%_{\text{noise}}$$

F_7: $P(t) \times= [1 - I(h)(1 - r_{\max})]$, where $I(h) = \max(0, \sin(\pi(h-6)/12))$ and $r_{\max} \sim U(0.4, 0.7)$.
F_8: Flat 7.2 kW per EV added during the calculated overnight charging window.
F_{10}: $P(t) \times= [1 + 0.05\, N_{\text{day}}]$ during $09:00 \leq h < 17:00$.

Step 5 – Rescale to Known Annual Total

Lines 433–445

```
1 if annual_kwh_override is not None and annual_kwh_override > 0:
2     profile *= annual_kwh_override / profile.sum()
3     ann_kwh  = annual_kwh_override
```

Magnitude Rescale

$$P_{\text{final}}(t) = P_{\text{scaled}}(t) \cdot \frac{E_{\text{target}}}{\sum_t P_{\text{scaled}}(t)}$$

This preserves the exact temporal shape while enforcing the correct annual total.

Table 2: Household load scaling factors

Factor	Name	Driver	Range	Type
F_1	Longitude	coastal/inland gradient, $\delta_c = \lambda - (-117.25)$	0.85–1.25	deterministic
F_2	Latitude	north/south step function at 32.60 / 32.70 / 32.85 / 32.95	0.90–1.10	deterministic
F_3	Elevation proxy	$1+0.15(\delta_c^+ + 2(\phi - 32.70)^+)$ representing inland cooling load growth	1.00–1.15	deterministic
F_4	Household size	occupancy scaling combined with appliance efficiency variation	0.70–1.30	deterministic
F_5	Density	Euclidean distance to city centre acting as urban density proxy	0.70–1.30	deterministic
F_6	Economic	coastal, northern, and urban income flags modifying baseline demand	0.95–1.25	deterministic
F_7	Solar adoption	sinusoidal daytime demand reduction representing rooftop PV penetration	0–0.60	stochastic
F_8	EV charging	flat overnight charging load added during nighttime hours	0 or 3–7 kW	deterministic
F_9	Multi-generation	extended family household multiplier for higher occupancy homes	1.0 or 1.2–1.5	stochastic
F_{10}	Daytime Occupants	Multiplier applied between 09:00 and 17:00 based on at-home residents	1.0–1.3+	deterministic

Section 4B – Synthetic Load Fallback (lines 465–486)

Lines 465–486

```python
def _build_annual_load_profile(annual_kwh, num_evs,
                               num_people, num_daytime_occupants):
    shape = np.array([
        0.40,0.35,0.32,0.30,0.30,0.35,  # 00-05 h  (night)
        0.50,0.70,0.85,0.80,0.75,0.72,  # 06-11 h  (morning ramp)
        0.70,0.68,0.65,0.65,0.70,0.90,  # 12-17 h  (afternoon)
        1.00,0.95,0.85,0.75,0.65,0.50,  # 18-23 h  (evening peak)
    ])
    for h in range(8760):
        sm = 1.20 if 6<=dt.month<=10 else (0.95 if dt.month in (3,4,5,11)
                                           else 1.00)
        om = 1.0 + 0.05*num_daytime_occupants if 9<=dt.hour<17 else 1.0
        ps = 0.70 + 0.10*min(num_people, 6)
        raw.append(shape[dt.hour] * sm * om * ps)
    scale = max(annual_kwh - ev_ann, 0.0) / arr.sum()
    return [raw[h]*scale + ev24[h % 24] for h in range(8760)]
```

Synthetic Load Formula

$$L(h) = s(h \bmod 24) \cdot S_m \cdot O_m(h) \cdot P_s \cdot \frac{E_{\text{annual}} - E_{\text{EV}}}{\sum_h s(h) \cdot S_m \cdot O_m(h) \cdot P_s} + P_{\text{EV}}(h \bmod 24)$$

where:

- $S_m \in \{0.95, 1.00, 1.20\}$: seasonal multiplier (spring/winter/summer)
- $O_m = 1 + 0.05\, N_{\text{day}}$ during 09–17 h, else 1.0

- $P_s = 0.70 + 0.10 \min(N_{\text{people}}, 6)$
- P_{EV}: 7.2 kW overnight for N_{EV} vehicles

Section 4C – Irradiance (lines 491–525)

Lines 491–525

```
def _irradiance_shape_factor(hour_of_day, day_of_year):
    sunrise = 6.0
    sunset  = 18.0 + 2.0*np.sin(2*np.pi*(day_of_year-80)/365.0)
    if hour_of_day < sunrise or hour_of_day >= sunset:
        return 0.0
    return max(0.0, np.sin(np.pi*(hour_of_day-sunrise)/(sunset-sunrise)))

def _fetch_irradiance(latitude, longitude):
    # Open-Meteo: 5-yr archive, field: shortwave_radiation_sum [MJ/m2/day]
    return round(np.mean(vals) / 3.6 * 365, 1)  # -> kWh/m2/yr
    # Fallback on any failure: 2080.0 kWh/m2/yr (San Diego 5-yr default)
```

Irradiance Models

Seasonal sunset hour:

$$h_{\text{set}}(d) = 18 + 2\sin\left(\frac{2\pi(d-80)}{365}\right) \quad \Rightarrow \quad h_{\text{set}} \in [16, 20]\ \text{h}$$

Hourly shape factor:

$$\Gamma(h, d) = \max\left(0,\ \sin\left(\frac{\pi(h-6)}{h_{\text{set}}(d) - 6}\right)\right) \quad \text{for } h \in [6,\ h_{\text{set}}),\ \text{else } 0$$

API unit conversion:

$$G_{\text{annual}} = \overline{G_{\text{daily}}}\ [\text{MJ/m}^2/\text{d}]\ \div\ 3.6\ \times\ 365 = G\ [\text{kWh/m}^2/\text{yr}]$$

Actual run value: $G = 1924.8\,\text{kWh/m}^2/\text{yr}$ (downtown SD).

Section 4D – SDG&E TOU Tariff Builder (lines 530–578)

Lines 530–578

```
def _build_hourly_tariffs(rate_plan, year=2024):
    # Reads SDG&E TOU CSV; assigns each of 8760 hours to a period
    for h in range(8760):
        on_p  = (16 <= hr < 21)
        sup_p = (hr < 6) or (mo in (3,4) and 10 <= hr < 14)
        # on_p  -> on_peak_$/kwh     (~$0.60)
        # sup_p -> super_off_peak    (~$0.26)
        # else  -> off_peak          (~$0.46)
    return out   # list[float], len=8760
```

TOU-DR Period Map

$$\pi(h,m) = \begin{cases} \pi_{\text{on}} & 16 \le h < 21 \\ \pi_{\text{super}} & h < 6 \ \text{ or } \ (m \in \{3,4\},\ 10 \le h < 14) \\ \pi_{\text{off}} & \text{otherwise} \end{cases}$$

Approximate 2024 rates: $\pi_{\text{on}} \approx \$0.60$, $\pi_{\text{off}} \approx \$0.46$, $\pi_{\text{super}} \approx \$0.26$/kWh. Average $\bar{\pi} \approx \$0.495$/kWh. The MILP uses the full 8760-element $\boldsymbol{\pi}$ vector in the import cost term of its objective, so it inherently schedules battery discharge for on-peak hours.

Section 4E & 4F – PV Output and Hardware Selection (lines 583–620)

Lines 583–620

```
583 def _build_hourly_pv_output(panel, n_panels, irradiance_kwh_m2_yr):
584     pkw = panel["panel_power_w"] / 1000.0
585     raw = [n_panels * pkw * Gamma(h%24, (h//24)%365+1) * PR
586            for h in range(8760)]
587     tgt = n_panels * pkw * irradiance_kwh_m2_yr * PR  # annual kWh target
588     return [v * tgt / sum(raw) for v in raw]           # normalised
589
590 def _select_panel(panel_brand):
591     if panel_brand is None:
592         return max(catalog, key=lambda p: p["efficiency_percent"]
593                                           / p["cost_per_wp_usd"])
594
595 def _select_battery(required_kwh):
596     cands = [b for b in BATTERY_CATALOG if b["usable_capacity_kwh"]
597              >= required_kwh]
598     if not cands: cands = BATTERY_CATALOG  # stack fallback
599     return min(cands, key=lambda b: b["cost_usd"]
600                                     / b["usable_capacity_kwh"])
```

PV Generation Model

$$P_{\text{PV}}(t) = P_{\text{raw}}(t) \cdot \frac{E_{\text{expected}}}{\sum_t P_{\text{raw}}(t)}, \quad E_{\text{expected}} = N \cdot \frac{P_{\text{panel}}}{1000} \cdot G_{\text{annual}} \cdot \text{PR}$$

The normalisation step forces the annual kWh of the shape-factor model to match the irradiance-based total exactly, removing any error from the sine approximation.
For $N = 11$, REC Alpha Pure, $G = 1924.8$, PR=0.80:

$$E = 11 \times \frac{405}{1000} \times 1924.8 \times 0.80 = 6{,}872\,\text{kWh/yr}$$

Section 4G – Rule-Based Dispatch Simulation (lines 625–673)

Lines 625–673

```
625 def _run_dispatch_simulation(hourly_load_kw, hourly_pv_kw,
626                              hourly_tariffs, battery):
627     soc = cap * 0.5
628     for h in range(8760):
```

```
        net = hourly_load_kw[h] - hourly_pv_kw[h]
        if net < 0:            # surplus: charge battery, export remainder
            ckw    = min(maxc, (cap - soc) / eta_c, -net)
            soc   += ckw * eta_c
            export = -net - ckw;   gbuy = 0.0
        else:                  # deficit: discharge battery, import remainder
            dkw    = min(maxd, soc, net)
            soc   -= dkw
            gbuy   = net - dkw;    export = 0.0
        soc      = clip(soc, 0, cap)
        imp_cost += gbuy   * hourly_tariffs[h]
        exp_cred += export * NEM_EXPORT_CREDIT
```

Rule-Based Dispatch Equations

Let $\Delta(t) = L(t) - P_{\mathrm{PV}}(t)$.
Surplus hour ($\Delta < 0$):

$$C(t) = \min\left(P_{\mathrm{ch,max}},\ \frac{B - \mathrm{SOC}}{\eta_c},\ -\Delta\right), \quad \mathrm{SOC}(t{+}1) = \mathrm{SOC}(t) + C(t)\eta_c, \quad \mathrm{Exp}(t) = -\Delta - C(t)$$

Deficit hour ($\Delta \geq 0$):

$$D(t) = \min(P_{\mathrm{dis,max}},\ \mathrm{SOC}(t),\ \Delta), \quad \mathrm{SOC}(t{+}1) = \mathrm{SOC}(t) - D(t), \quad \mathrm{Imp}(t) = \Delta - D(t)$$

This function runs *after* the MILP, using the MILP-chosen hardware, to compute dollar-weighted import cost and export credit needed by the NPV model.

Section 4H – 10-Year NPV Model (lines 678–746)

Lines 678–746

```
def _compute_economics(dispatch, panel, n_panels, battery,
                       battery_units, annual_load_kwh,
                       avg_tariff, with_battery):
    gross   = (pv_cost + bat_cost) * (1 + 0.10)   # +10% install
    net_cap = gross * (1 - 0.30)                   # -30% ITC

    for y in range(1, 11):
        inf  = (1.06)**(y-1)
        disc = (1.07)**y
        scl  = (1 - panel["degradation_rate"])**(y-1)  # PV decay
        imp  = annual_load_kwh - (annual_load_kwh
               - dispatch["annual_import_kwh"]) * scl
        exp  = dispatch["annual_export_kwh"] * scl
        sol  = imp*avg_tariff*inf - exp*NEM_CREDIT + fixed + om + inv
        npv += (trad - sol) / disc
```

10-Year NPV Formulas

Year-0 net CAPEX:

$$C_{\mathrm{net}} = (C_{\mathrm{PV}} + C_{\mathrm{bat}}) \times 1.10 \times 0.70$$

PV degradation scaling:

$$\sigma_y = (1 - d)^{y-1}, \quad d = 0.005\ (\mathrm{REC})$$

$$E_{\text{import}}(y) = E_L - (E_L - E_{\text{imp},0})\sigma_y \quad \text{(grows as PV decays)}, \quad E_{\text{export}}(y) = E_{\text{exp},0} \cdot \sigma_y$$

Annual solar cost:

$$C_{\text{solar}}(y) = E_{\text{imp}}(y) \cdot \bar{\pi} \cdot 1.06^{y-1} - E_{\text{exp}}(y) \cdot \pi_{\text{NEM}} + C_{\text{fixed}} + C_{\text{O\&M}} \cdot 1.06^{y-1} + \$2000 \cdot \mathbf{1}_{y=10}$$

NPV and payback:

$$\text{NPV} = -C_{\text{net}} + \sum_{y=1}^{10} \frac{C_{\text{trad}}(y) - C_{\text{solar}}(y)}{1.07^y}, \qquad y^* = \min\Big\{y : \sum_{k=1}^{y} \big[C_{\text{trad}}(k) - C_{\text{solar}}(k)\big] \geq C_{\text{net}}\Big\}$$

Verified run results:

Scenario	**Net CAPEX**	**Yr-1 Bill**	**Payback**	**NPV**
PV only	\$3,555	\$7,808	3 yr	\$6,428
PV + Battery	\$12,410	\$7,805	9 yr	-\$2,397

Section 5 – Core MILP Optimizer (lines 753–981)

Variable Layout

Lines 838–845 – variable index layout

```
n_vars = 5*T + 3              # T=8760 -> 43,803 total variables
I_P   = 0                     # N  Panels  (INTEGER)
I_B   = 1                     # B  BatCap  (continuous kWh)
I_IMP = np.arange(2,        T+2)      # Import(t)      8760 values
I_EXP = np.arange(T+2,    2*T+2)     # Export(t)      8760 values
I_CHG = np.arange(2*T+2, 3*T+2)     # Charge(t)      8760 values
I_DIS = np.arange(3*T+2, 4*T+2)     # Discharge(t)   8760 values
I_SOC = np.arange(4*T+2, 5*T+3)     # SOC(t)         8761 values (t=0..T)
```

Decision Variable Summary

Index	Variable	Type	Count	Bounds
0	N Panels	**Integer**	1	$[0,\ A_{\text{roof}}/A_p]$
1	B BatCap	Continuous	1	$[0,\ 50]$ kWh
$2\ldots T{+}1$	Imp(t)	Continuous	8760	$[0,\ \infty)$
$T{+}2\ldots 2T{+}1$	Exp(t)	Continuous	8760	$[0,\ \infty)$
$2T{+}2\ldots 3T{+}1$	$C(t)$ Charge	Continuous	8760	$[0,\ \infty)$
$3T{+}2\ldots 4T{+}1$	$D(t)$ Discharge	Continuous	8760	$[0,\ \infty)$
$4T{+}2\ldots 5T{+}2$	SOC(t)	Continuous	8761	$[0,\ 200]$
	Total		**43,803**	

The single integer variable N makes this a MILP. HiGHS solves the LP relaxation then uses branch-and-bound.

Objective Function

Lines 850–854 – objective vector

```
c_obj[I_P]   = panel_cost * (1 - FEDERAL_ITC_RATE) * ANNUITY_FACTOR
c_obj[I_B]   = BAT_COST_PER_KWH_PROXY * (1 - FEDERAL_ITC_RATE) * ANNUITY_FACTOR
c_obj[I_IMP] = Pbuy           # 8760-element TOU price vector
c_obj[I_EXP] = -NEM_EXPORT_CREDIT    # = -0.10  (revenue from export)
# c_obj[I_CHG] = c_obj[I_DIS] = c_obj[I_SOC] = 0  (internal variables)
```

Objective – Minimise Annualised Total Cost

$$\min_{N,B,\text{Imp},\text{Exp},C,D,\text{SOC}} \underbrace{(N\,C_P + B\,c_B)(1-\text{ITC})\,\alpha}_{\text{annualised CAPEX}} + \underbrace{\sum_{t=1}^{T} \pi(t)\,\text{Imp}(t)}_{\text{import bill}} - \underbrace{\pi_{\text{NEM}} \sum_{t=1}^{T} \text{Exp}(t)}_{\text{NEM credit}}$$

Annualised coefficients for this run:

$$c_N = 405\times 2.85\times 0.70\times 0.1424 = \$113.55\ /\text{panel/yr}, \quad c_B = 852\times 0.70\times 0.1424 = \$84.84\ /\text{kWh/yr}$$

All 10 Constraint Groups

Lines 884–928 – constraint assembly

```
# C1  Energy balance equality (8760 rows)
eq([(I_P, pkw*irr_h[t]), (I_IMP[t],1), (I_EXP[t],-1),
    (I_CHG[t],-1),        (I_DIS[t],1)],  Load[t])

# C2  SOC dynamics equality (8760 rows)
eq([(I_SOC[t],-1), (I_SOC[t+1],1),
    (I_CHG[t],-ETA_C), (I_DIS[t],1/ETA_D)],  0.0)

# C0  SOC initial condition (1 row)
eq([(I_SOC[0],1.0), (I_B,-SOC_INIT_FRAC)],  0.0)

# C3  SOC <= BatCap                  (8761 rows)
# C4  SOC >= 0.10 * BatCap (DoD)    (8761 rows)
# C5  Charge  <= 0.50 * BatCap      (8760 rows)
# C6  Discharge <= 0.50 * BatCap    (8760 rows)
# C7  Export <= PV generated         (8760 rows)
# C8  N * panel_area <= roof_area    (1 row)
# C9  (N*C_P + B*c_B) * 1.10 <= budget  (1 row)
```

Constraints in Mathematical Form

$$\text{C1:}\quad NP_{\text{kW}}G_h(t) + D(t) + \text{Imp}(t) = L(t) + C(t) + \text{Exp}(t) \quad \forall t \tag{1}$$

$$\text{C2:}\quad \text{SOC}(t{+}1) = \text{SOC}(t) + \eta_c C(t) - D(t)/\eta_d \quad \forall t \tag{2}$$

$$\text{C0:}\quad \text{SOC}(0) = 0.50\,B \tag{3}$$

$$\text{C3:}\quad \text{SOC}(t) \le B \quad \forall t \tag{4}$$

$$\text{C4:}\quad \text{SOC}(t) \ge 0.10\,B \quad \forall t \tag{5}$$

$$\text{C5:}\quad C(t) \le 0.50\,B \quad \forall t \tag{6}$$

$$\text{C6:}\quad D(t) \le 0.50\,B \quad \forall t \tag{7}$$

$$\text{C7:}\quad \text{Exp}(t) \le NP_{\text{kW}}G_h(t) \quad \forall t \tag{8}$$

$$\text{C8:}\quad NA_p \le A_{\text{roof}} \tag{9}$$

$$\text{C9:}\quad (NC_P + Bc_B) \times 1.10 \le C_{\text{budget}} \tag{10}$$

Row counts: C1 + C2: $2 \times 8760 = 17{,}520$; C3 + C4: $2 \times 8761 = 17{,}522$; C5 + C6 + C7: $3 \times 8760 = 26{,}280$; C0 + C8 + C9: 3. Total: **61,325** constraint rows.
Sparse matrix: $A \in \mathbb{R}^{61325\times 43803}$, nnz = 166,449, density = 0.006%. COO → CSC for HiGHS. Solve at `mip_rel_gap=0.005`: stops when the best integer solution is within 0.5% of the LP lower bound.

Section 6 – Master Orchestrator (lines 988–1097)

Lines 988–1097 – 9-step pipeline

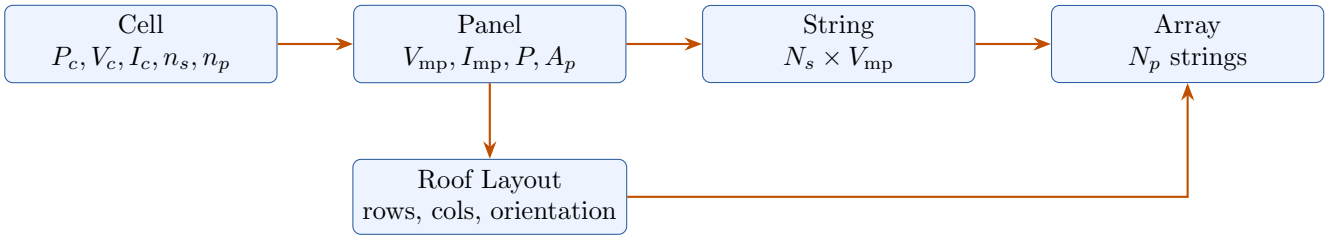
```python
def recommend_residential_energy_system(user_profile: dict) -> dict:
    # Step 1 -- Validate and extract
    lat, lon, bud, roof = ...
    # NEW: also reads roof_length_m, roof_width_m from user_profile
    # (acfg() closure reads user_profile directly)

    # Step 2 -- Irradiance      -> irr = 1924.8 kWh/m2/yr
    # Step 3 -- Tariffs         -> tars [8760 floats], avgt = $0.4951/kWh
    # Step 4 -- Load            -> load [8760 kW], akwh = 7500
    # Step 5 -- Panel           -> REC Alpha Pure (22.6%, $2.85/Wp)
    # Step 5.5- True Geometry   -> Max panels = 4, constrains roof area
    # Step 6 -- MILP PV-only    -> N*=4, B*=0,      Optimal, 18.1 s
    # Step 7 -- MILP PV+bat     -> N*=4, B*=5.0,    Optimal, 13.5 s
    # Step 8 -- Hourly PV arrays for both scenarios
    # Step 9 -- Dispatch sim + 10-yr economics for both scenarios
    #           Battery selected: Tesla Powerwall 3, 1 unit
```

Pipeline Data Flow

Step	Function / Action	Key output	Feeds
1	Validate `USER_PROFILE`	lat, lon, roof, bud, ℓ_r, w_r	all steps
2	`_fetch_irradiance`	$G = 1924.8$ kWh/m^2/yr	MILP, PV gen
3	`_build_hourly_tariffs`	$\boldsymbol{\pi}$ [8760]	MILP obj, economics
4	`_load_..._eia`	$\mathbf{L}$ [8760 kW]	MILP C1, economics
5	`_select_panel`	panel dict	MILP, PV gen, **acfg**
5.5	Calculate True Physical Fit	max_panels, limited A_{roof}	MILP C8
6	MILP PV-only	$N^* = 4$	Step 8, 9
7	MILP PV+Battery	$N^* = 4$, $B^* = 5.0$ kWh	Step 8, 9
8	`_build_hourly_pv_output`	$\mathbf{P}_{\text{PV}}$ [8760]	dispatch sim
9	dispatch + economics	NPV, payback, costs	output dict
	`acfg`(N^*)	**full layout dict**	output dict

Section 7 – Output Assembly: `acfg()` Full Array Configuration

The `acfg()` (v3, ∼55 lines) returns 14 keys spanning three levels:



Full `acfg()` Function

Correct version of lines 1108–1199 (8-space body indent)

```
1     def acfg(n):
2         """
3         Full array configuration:
4           1. Try portrait and landscape panel orientations.
5           2. For each layout, find string split closest to 400 V.
6           3. Return orientation with highest roof utilisation.
7         """
8         # Panel dimensions estimated from area + 0.75 aspect shorthand
9         p_w = panel["panel_area_m2"] ** 0.5 * 0.75    # approx width  ~1.0 m
10        p_h = panel["panel_area_m2"] / p_w             # approx height ~1.8 m
11
12        # Roof dims from USER_PROFILE, else sqrt 2:1 fallback
13        rl = float(user_profile.get("roof_length_m") or math.sqrt(roof*2))
14        rw = float(user_profile.get("roof_width_m")  or math.sqrt(roof/2))
15
16        best = None;  best_util = -1.0
17
18        for (ph, pw) in [(p_h, p_w), (p_w, p_h)]:   # portrait, landscape
19            orientation = "portrait" if ph > pw else "landscape"
20            cols = max(1, int(rw // pw))   # panels across roof width
21            rows = max(1, int(rl // ph))   # panels along roof length
22            if rows * cols < n:
23                continue   # orientation cannot fit n panels
24
25            # Electrical: minimise |N_s * Vmp - 400| over divisors of n
26            b_ns, b_np, b_diff = 1, n, 1e9
27            for ns2 in range(1, n+1):
28                if n % ns2 != 0: continue
29                diff = abs(ns2 * panel["vmp_v"] - 400)
30                if diff < b_diff:
31                    b_diff=diff; b_ns=ns2; b_np=n//ns2
32
33            util = round(100 * n * panel["panel_area_m2"] / roof, 1)
34            candidate = {
35                "orientation":                    orientation,
36                "panels_per_row_on_roof":         cols,
37                "panel_rows_on_roof":             rows,
38                "max_panels_fitting_roof":        rows * cols,
39                "cells_in_series_per_panel":      panel["cells_in_series"],
40                "cells_in_parallel_per_panel":    panel["cells_in_parallel"],
41                "panels_in_series_per_string":    b_ns,
42                "parallel_strings":               b_np,
43                "total_panels":                   n,
44                "string_voltage_v":        round(b_ns * panel["vmp_v"], 2),
```

```
                "array_current_a":        round(b_np * panel["imp_a"], 2),
                "total_dc_capacity_kw":   round(n * panel["panel_power_w"]/1000, 3),
                "total_array_area_m2":    round(n * panel["panel_area_m2"], 2),
                "roof_area_utilization_percent": util,
            }
            if util > best_util:
                best_util = util;  best = candidate

        if best is None:   # fallback: no roof dims supplied
            b_ns, b_np, b_diff = 1, n, 1e9
            for ns2 in range(1, n+1):
                if n % ns2 != 0: continue
                diff = abs(ns2*panel["vmp_v"]-400)
                if diff < b_diff: b_diff=diff; b_ns=ns2; b_np=n//ns2
            best = {
                "orientation":                    "undetermined",
                "panels_per_row_on_roof":         None,
                "panel_rows_on_roof":             None,
                "max_panels_fitting_roof":        None,
                "cells_in_series_per_panel":      panel["cells_in_series"],
                "cells_in_parallel_per_panel":    panel["cells_in_parallel"],
                "panels_in_series_per_string":    b_ns,
                "parallel_strings":               b_np,
                "total_panels":                   n,
                "string_voltage_v":        round(b_ns*panel["vmp_v"],2),
                "array_current_a":         round(b_np*panel["imp_a"],2),
                "total_dc_capacity_kw":    round(n*panel["panel_power_w"]/1000,3),
                "total_array_area_m2":     round(n*panel["panel_area_m2"],2),
                "roof_area_utilization_percent":
                    round(100*n*panel["panel_area_m2"]/roof,1),
            }
        return best

    def bcfg(spec, units, _kwh):   # 4-space indent -- peer of acfg, NOT inside it
        ...
```

Mathematical Foundation

Panel Dimension Estimation from Area

Approximate Panel Width and Height

Catalog entries store `panel_area_m2` but not explicit dimensions. We estimate them from area using a fixed shorthand coefficient:

$$w_p \approx \sqrt{A_p} \times 0.75, \qquad h_p = \frac{A_p}{w_p}$$

For REC Alpha Pure ($A_p = 1.79\,\mathrm{m}^2$):

$$w_p \approx \sqrt{1.79} \times 0.75 = 1.338 \times 0.75 = 1.003\,\mathrm{m}, \qquad h_p = \frac{1.79}{1.003} = 1.784\,\mathrm{m}$$

Landscape simply swaps: $w_p^L = h_p = 1.784\,\mathrm{m}$, $h_p^L = w_p = 1.003\,\mathrm{m}$.
Accuracy: $\pm 5\%$ for all 60/66-cell monocrystalline panels in the catalog. To use exact manufacturer dimensions, add `"panel_width_m"` and `"panel_height_m"` keys to each catalog entry and replace the two lines that compute `p_w` and `p_h`.

Physical Layout: Rows and Columns on the Roof

Portrait vs Landscape Panel Count

Given roof dimensions (ℓ_r, w_r) from `USER_PROFILE` and panel face dimensions (h_p, w_p):
Portrait ($h_p > w_p$, panel stands tall):

$$N_{\text{cols}}^P = \left\lfloor \frac{w_r}{w_p} \right\rfloor, \quad N_{\text{rows}}^P = \left\lfloor \frac{\ell_r}{h_p} \right\rfloor, \quad N_{\text{max}}^P = N_{\text{cols}}^P \times N_{\text{rows}}^P$$

Landscape ($h_p < w_p$ after axis swap):

$$N_{\text{cols}}^L = \left\lfloor \frac{w_r}{h_p} \right\rfloor, \quad N_{\text{rows}}^L = \left\lfloor \frac{\ell_r}{w_p} \right\rfloor, \quad N_{\text{max}}^L = N_{\text{cols}}^L \times N_{\text{rows}}^L$$

For REC Alpha Pure on $\ell_r = 3.0\,\mathrm{m}$, $w_r = 5.0\,\mathrm{m}$:

Orientation	N_{cols}	N_{rows}	N_{max}	Fits $N^* = 4$?
Portrait	$\lfloor 5.0/1.003 \rfloor = 4$	$\lfloor 3.0/1.784 \rfloor = 1$	4	✓
Landscape	$\lfloor 5.0/1.784 \rfloor = 2$	$\lfloor 3.0/1.003 \rfloor = 2$	4	✓

Both orientations max out at exactly 4 panels. Utilisation is 100.0% for both, so portrait (evaluated first in the loop) is returned.

Cell Topology Inside Each Panel

Within-Panel Cell Configuration

n_s (cells in series) and n_p (parallel cell sub-strings) are read directly from the manufacturer catalog and passed through unchanged:

$$P_c = \frac{P_{\text{panel}}}{n_s n_p}, \quad V_c = \frac{V_{\text{mp}}}{n_s}, \quad I_c = \frac{P_c}{V_c}, \quad V_{\text{oc},c} = \frac{V_{\text{oc}}}{n_s}, \quad I_{\text{sc},c} = \frac{I_{\text{sc}}}{n_p}$$

For REC Alpha Pure ($n_s = 60$, $n_p = 1$, $P = 405\,\mathrm{W}$, $V_{\text{mp}} = 36.4\,\mathrm{V}$, $I_{\text{mp}} = 13.1\,\mathrm{A}$):

$$P_c = 6.75\,\mathrm{W}, \quad V_c = 0.607\,\mathrm{V}, \quad I_c = 11.12\,\mathrm{A}, \quad V_{\text{oc},c} = 0.720\,\mathrm{V}, \quad I_{\text{sc},c} = 13.8\,\mathrm{A}$$

All 9 catalog panels have $n_p = 1$ (standard for monocrystalline PERC/TOPCon/HJT). The field

is retained for future half-cut or multi-busbar panels.

Electrical String Wiring

String Split Optimisation

Residential string inverters have MPPT input range $\approx$ 350–500 V. Target: 400 V.
Find the divisor pair (N_s, N_p) with $N_s \cdot N_p = N^*$ minimising $|N_s V_{\text{mp}} - 400|$:

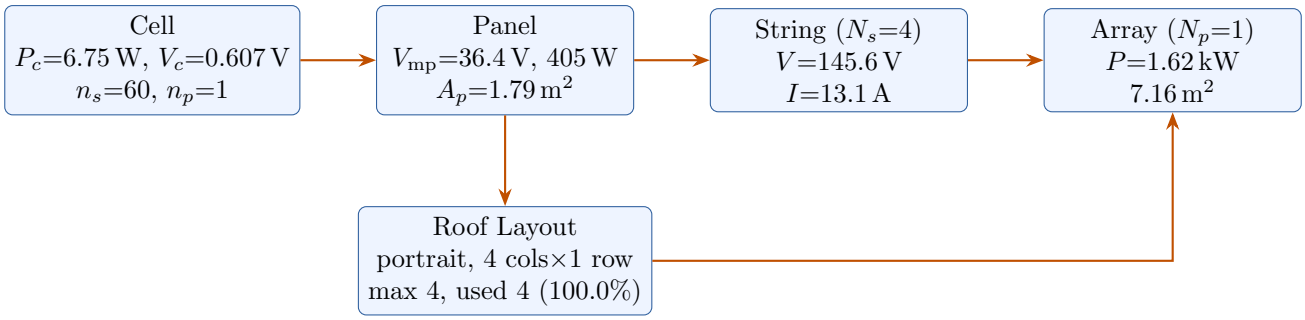
$$(N_s^*, N_p^*) = \arg \min_{\substack{N_s N_p = N^* \\ N_s | N^*}} \left| N_s \cdot V_{\text{mp}} - 400 \right|$$

For $N^* = 4$, $V_{\text{mp}} = 36.4$ V. Divisors of 4: $\{1, 2, 4\}$.

N_s	N_p	V_{string}	$\lvert V - 400 \rvert$
1	4	$1 \times 36.4 = 36.4$ V	363.6
2	2	$2 \times 36.4 = 72.8$ V	327.2
4	1	$4 \times 36.4 = 145.6$ V	**254.4** $\leftarrow$ selected

$$V_{\text{array}} = N_s^* \cdot V_{\text{mp}} = 145.6\,\text{V}, \qquad I_{\text{array}} = N_p^* \cdot I_{\text{mp}} = 1 \times 13.1 = 13.1\,\text{A}, \qquad P_{\text{DC}} = 1.62\,\text{kW}$$

Full Hierarchy and Output



Expected console output for $N^* = 4$, REC Alpha Pure, 3.0 m × 5.0 m roof

```
Array Configuration
---------------------------------------------
orientation                                    portrait
panels_per_row_on_roof                         4
panel_rows_on_roof                             1
max_panels_fitting_roof                        4
cells_in_series_per_panel                      60
cells_in_parallel_per_panel                    1
panels_in_series_per_string                    4
parallel_strings                               1
total_panels                                   4
string_voltage_v                               145.6
array_current_a                                13.1
total_dc_capacity_kw                           1.62
total_array_area_m2                            7.16
roof_area_utilization_percent                  100.0
```

Output field	Formula / meaning
orientation	Portrait (tall) or landscape (wide)
panels_per_row_on_roof	$N_{\text{cols}} = \lfloor w_r/w_p \rfloor$
panel_rows_on_roof	$N_{\text{rows}} = \lfloor \ell_r/h_p \rfloor$
max_panels_fitting_roof	$N_{\text{cols}} \times N_{\text{rows}}$
cells_in_series_per_panel	n_s from catalog
cells_in_parallel_per_panel	n_p from catalog
panels_in_series_per_string	N_s^* minimising $\lvert N_s V_{\text{mp}} - 400 \rvert$
parallel_strings	$N_p^* = N^*/N_s^*$
total_panels	N^* from MILP
string_voltage_v	$N_s^* \cdot V_{\text{mp}}$
array_current_a	$N_p^* \cdot I_{\text{mp}}$
total_dc_capacity_kw	$N^* \cdot P_{\text{panel}}/1000$
total_array_area_m2	$N^* \cdot A_p$
roof_area_utilization_percent	$N^* A_p / A_r \times 100$

Appendix – All Constants Quick Reference

Constant	Value	Meaning
SDGE_TOTAL_CUSTOMERS	1,040,149	All SDG&E meters incl. commercial
COASTAL_LON_REF	−117.25°	Coastal/inland split for F1
CITY_CENTER_LAT	32.7157°	Downtown San Diego – F5
CITY_CENTER_LON	−117.1611°	Downtown San Diego – F5
G_REF_W_M2	1000 W/m^2	STC reference irradiance
PR_PERFORMANCE_RATIO	0.80	System losses factor
INSTALLATION_COST_RATE	10%	Labour on hardware cost
FEDERAL_ITC_RATE	30%	Federal ITC 2025
UTILITY_INFLATION_RATE	6%/yr	SDG&E bill escalation
DISCOUNT_RATE	7%/yr	NPV cost of capital
O_AND_M_COST_PER_W_YR	\$0.005/W/yr	Cleaning + insurance
NEM_EXPORT_CREDIT	\$0.10/kWh	NEM 3.0 avoided-cost
INVERTER_REPLACEMENT_USD	\$2,000	Year-10 inverter swap
ANALYSIS_YEARS	10	Financial horizon
SDGE_DAILY_FIXED_FEE	\$0.345/day	SDG&E daily minimum charge
ANNUITY_FACTOR	0.1424	r=7%, n=10 yr
BAT_COST_PER_KWH_PROXY	\$852/kWh	Powerwall 3: \$11,500/13.5 kWh
MILP_ETA_C	0.975	Charge efficiency η_c
MILP_ETA_D	0.975	Discharge efficiency η_d
MILP_C_RATE	0.50	Max 50% capacity/hr
MILP_SOC_MIN_FRAC	0.10	Depth-of-discharge floor
MILP_SOC_INIT_FRAC	0.50	Initial SOC = 50% of B
EV_CHARGE_START_HOUR	22:00	EV charging window start
EV_CHARGE_END_HOUR	06:00	EV charging window end
EV_CHARGER_POWER_KW	7.2 kW	Level-2 EVSE power
EV_DAILY_ENERGY_KWH	14.0 kWh	Energy per EV per day